ResearchOS

Research Orchestration System | Interview revision guide

> **In one sentence:** ResearchOS finds and organizes arXiv papers, indexes their PDFs, and answers focused research questions using passages retrieved from those PDFs.

What a user can do

- Create projects with separate paper libraries and saved conversations.
- Search arXiv by a paper title or ID, or discover work by topic.
- Choose paper Q&A, summarization, results extraction, comparison, or literature synthesis.
- Ask general questions using supplied project and recent-chat context without paper retrieval.

One request, from browser to answer

1. Choose an action	The browser action selector makes the task explicit. It adds a short prefix to the request; it does not send a raw tool ID.
2. Add context	FastAPI loads project metadata and up to 10 earlier messages from the current chat. It can resolve clear follow-ups such as "summarize it".
3. Route one tool	The backend routes the request to one matching tool. Tool calls are not run in parallel, which helps keep actions separate.
4. Prepare evidence	If a content task needs a new paper, the backend finds and saves it, then tries to index its PDF before retrieval.
5. Answer and save	The chosen workflow retrieves paper- and project-scoped chunks, generates an answer from those excerpts, and saves it to the chat.

Main technologies

Python and FastAPI for the API; **OpenAI API** for function calling, response generation, and embeddings; **arXiv API** for paper metadata; **LlamaIndex** for PDF reading, chunking, and vector-store integration; **PostgreSQL and pgvector** for metadata and semantic search; **HTML, CSS, and JavaScript** for the browser app.

This is a learning project with one API service and a local database setup, not a production-hardened service.

Architecture choices and lessons

Tools and handlers

The model sees six callable functions as **tools**: paper search, focused Q&A;, summarization, results extraction, comparison, and synthesis. `main.py` defines their schemas and dispatches calls. The Python modules in `app/tools/` implement the content operations and shared helpers. These are ordinary server-side functions, with no separate agent framework.

Callable names

`search_and_save_papers` — discovery and saving; `answer_paper_question` — focused Q&A; `summarize_saved_paper` — summary; `extract_saved_paper_results` — results; `compare_saved_papers` — comparison; `synthesize_saved_literature` — multi-paper synthesis.

One model tool call can still run several internal steps. For example, a summary tool handler retrieves paper excerpts and then calls the model to write the summary. The frontend action selector makes intent explicit, and the backend constrains the model to one matching tool when possible.

Why these choices?

Explicit action choice	Similar requests can overlap, such as summary versus result extraction. A simple selector clarifies intent; the server limits a request to one tool call.
SQL plus vector storage	SQL tables store projects, papers, and chats. pgvector supports similarity search over extracted paper chunks.
Paper and project filters	Retrieval uses both IDs so one paper or project cannot supply evidence for another.
Ingest before retrieval	A new paper is saved and indexed before a content workflow uses it. Metadata and abstracts are not treated as evidence from the paper.
Context without RAG	Recent chat and project metadata help with simple context questions. Vector search is used when an answer needs paper passages.
Small first version	One FastAPI app, a static frontend, and PostgreSQL kept the system manageable for a first LLM engineering project.

Difficulties we faced

Wrong paper match	A broad arXiv query for a named work once returned a paper that cited it. Title lookup was separated from topic discovery to make named-paper requests stricter.
Overlapping actions	We tried prompt-only selection, but it could confuse adjacent tasks. We returned to a simple UI action selector; the selected action is phrased in the request and the backend routes to one tool. Better automatic selection plus an evaluation set is future work.
Ingestion latency	Indexing extracts PDF text, chunks it, creates embeddings, and writes them to PostgreSQL. It takes longer than a normal chat turn. Query embeddings are separate and are used during retrieval.
PDF extraction edge case	One PDF produced an ingestion error mentioning NUL (0x00) characters. The pipeline does not clean extracted text or switch parsers yet; sanitization or a fallback extractor remains future work.
Frontend polish	Long conversations exposed scrolling issues, and equations needed MathJax rendering and a modest size adjustment. These were presentation fixes, separate from the RAG workflow.

Interview lesson: keep discovery, ingestion, retrieval, and generation distinct. This makes failures easier to explain and clarifies what the model has actually read.

Codebase guide

All code, configuration, and test files used by the app are listed here. The guide itself and local-only .env and virtual-environment files are excluded.

Root and startup files

File	Purpose
`README.md`	Project overview, architecture, setup, request flow, design choices, and current limits.
`.gitignore`	Excludes the virtual environment, local .env secrets, Python cache files, and macOS metadata from Git.
`docker-compose.yml`	Starts PostgreSQL with pgvector and a persistent local data volume.
`backend/requirements.txt`	Lists the Python packages used by the API, database layer, OpenAI client, arXiv requests, and LlamaIndex integrations. Install this file in the project virtual environment before starting the app.
`backend/create_tables.py`	Imports the SQLAlchemy models and creates their relational tables in the configured database. Run it once after starting PostgreSQL and setting DATABASE_URL.
`backend/test.py`	A manual comparison example that calls the comparison handler with hard-coded paper and project IDs. Replace the sample IDs with records in your own database before running it.

API and data model

File	Purpose
`backend/app/__init__.py`	Marks app as a Python package.
`backend/app/main.py`	The central API and tool router. It defines the six OpenAI function schemas, maps names to Python functions, loads chat/project context, prepares papers, chooses one tool, and saves final responses.
`backend/app/database.py`	Loads the local environment and DATABASE_URL, then creates the SQLAlchemy engine, session factory, and shared declarative base used by every database model.
`backend/app/models.py`	Defines tables for projects, papers, project-paper links, chat sessions, chat messages, and user profiles. Relationships connect a project to its papers and chats.
`backend/app/schemas.py`	Defines and validates the JSON accepted and returned by API endpoints, including the research question, project/chat IDs, paper metadata, and chat messages.

Services

File	Purpose
`backend/app/services/__init__.py`	Marks the services directory as a Python package. It contains integrations and data helpers used by the API.
`backend/app/services/arxiv.py`	Normalizes modern and legacy arXiv IDs, calls the arXiv Atom API, and parses titles, abstracts, authors, dates, and links. It keeps exact-title lookup separate from broader topic discovery.
`backend/app/services/papers.py`	Persists arXiv metadata in the papers table and creates the project-paper link. It reuses a paper already stored by arXiv ID instead of making a duplicate.
`backend/app/services/context_builder.py`	Builds the context object sent into request routing by combining the user profile, current project data, and recent messages from the selected chat.
`backend/app/services/user_context.py`	Loads the local user profile and merges updates to preferences, interests, and expertise. The context builder uses these values for general context questions.
`backend/app/services/test_arxiv.py`	A manual network smoke test for arXiv ID and title lookup. It checks a known paper and prints returned metadata; it contacts the live arXiv service.

Codebase guide: evidence and workflows

RAG and vector storage

File	Purpose
`backend/app/rag/__init__.py`	Marks the RAG directory as a Python package. The RAG modules handle reading, indexing, storing, and retrieving paper text.
`backend/app/rag/answer.py`	Implements focused Q&A; for one paper: it retrieves relevant chunks, sends those excerpts to OpenAI, and returns a grounded answer with page-level source records.
`backend/app/rag/ingestion.py`	Downloads a PDF with a size limit, extracts one document per page, adds paper/project/page metadata, splits the text into overlapping chunks, creates embeddings, and stores them in pgvector.
`backend/app/rag/retrieval.py`	Creates a vector retriever for a question and applies both paper ID and project ID filters. It returns the matched text, page/arXiv metadata, and similarity score.
`backend/app/rag/vector_store.py`	Reads PostgreSQL connection details from DATABASE_URL and creates cached OpenAI embedding and pgvector objects. The same configuration is shared by ingestion and retrieval.
`backend/app/rag/test_answer.py`	Manual example for asking a question about an already-indexed paper. It prints the answer and the pages used; the hard-coded IDs must exist locally.
`backend/app/rag/test_ingestion.py`	Manual end-to-end ingestion script. It reads TEST_PAPER_ID, TEST_PROJECT_ID, TEST_ARXIV_ID, and TEST_PDF_URL from the environment and contacts the PDF and database.
`backend/app/rag/test_retrieval.py`	Manual semantic-retrieval example with sample paper/project IDs. It prints retrieved pages, scores, and text snippets for inspection.
`backend/app/rag/test_vector_store.py`	Manual initialization check for database configuration, pgvector setup, and embedding model setup. It requires the local environment variables and reachable database.

Research workflow handlers

File	Purpose
`backend/app/tools/__init__.py`	Exports the summary, result-extraction, comparison, and synthesis handlers so main.py can register and call them.
`backend/app/tools/evidence.py`	Shared evidence helpers: retrieve chunks for one paper, retrieve each paper separately in multi-paper tasks, format labeled excerpts, and deduplicate source pages.
`backend/app/tools/generate.py`	Shared OpenAI Responses API helper for generating text from supplied excerpts. It adds grounding and TeX-formatting instructions used by content tools.
`backend/app/tools/summarisation.py`	Implements the summary operation. It retrieves likely relevant parts of one paper and asks the model for a structured summary with page citations.
`backend/app/tools/results_extraction.py`	Implements results extraction. It retrieves evaluation passages, optionally applies a user focus, and asks the model to preserve reported numbers and cite pages.
`backend/app/tools/paper_comparison.py`	Implements comparison for two or more papers. It retrieves evidence paper-by-paper, rejects invalid paper lists, and asks the model to keep claims attributed to the correct source.
`backend/app/tools/literature_synthesis.py`	Implements multi-paper synthesis on a required topic. It gathers labeled evidence and asks for shared themes, disagreements, complementary findings, and open questions.
`backend/app/tools/test_tools.py`	Mock-based unit tests for the content handlers. They cover missing evidence, input requirements, grounded outputs, and source results without making model calls.
`backend/app/test_search_and_save.py`	Mock-based unit tests for the search-and-save path. They check new and already-indexed papers, missing PDFs, failed ingestion, and mixed outcomes.

Codebase guide: frontend

File	Purpose
`frontend/index.html`	Defines the workspace and chat navigation, paper library, action selector, composer, and MathJax configuration. It also loads the JavaScript and stylesheet.
`frontend/app.js`	Calls the API and keeps current project/chat/message state. It renders the library and conversation, turns the selected action into a request prefix, handles errors, and typesets equations.
`frontend/styles.css`	Defines the black-and-white glass visual style and responsive layouts. It controls scrolling panels, paper cards, chat messages, mobile menus, and equation readability.

A one-minute project explanation

> "ResearchOS is a research assistant that organizes papers by project and answers questions from their PDFs. A user chooses a research action in the browser. FastAPI routes it to one tool, searches arXiv if a needed paper is missing, saves and indexes the PDF, then retrieves chunks filtered by paper and project. A focused workflow generates a response from those excerpts, and the app stores the result in chat history."

Interview questions to practice

- **Why RAG?** Paper-specific answers need evidence from the paper, not only model memory. Retrieval supplies passages and page information.

- **Why SQL plus pgvector?** SQL stores project and paper records; vector search finds relevant PDF passages.

- **How do you prevent cross-paper answers?** Retrieval filters on both paper ID and project ID; multi-paper workflows label excerpts by paper.

- **How do you reduce tool overlap?** The user selects an action; the frontend makes intent explicit and the backend narrows the tool choice to one operation.

- **What would you improve?** Add routing/retrieval evaluations, make ingestion asynchronous, and handle unusual PDF text with cleaning or a fallback parser.

Be precise: this is project-scoped RAG with function calling, not a multi-agent system. The action selector is a deliberate first version; automatic routing can be improved and measured later.

Run ResearchOS locally

Prerequisites: Python 3.10 or later, Docker Compose, and an OpenAI API key.

1. Create and activate a virtual environment from the project root

```
python3 -m venv .venv
source .venv/bin/activate
```

2. Install the backend dependencies

```
pip install -r backend/requirements.txt
```

3. Start the local database

```
docker compose up -d postgres
```

4. Put your settings in a root .env file

```
OPENAI_API_KEY=your_openai_api_key
DATABASE_URL=postgresql://researchos:researchos@localhost:5432/researchos
```

5. Create the relational tables and start the API

```
cd backend
python create_tables.py
uvicorn app.main:app --reload
```

6. Open the app

```
http://localhost:8000/ui/
API reference: http://localhost:8000/docs
```

Local setup notes

- Keep .env local; .gitignore excludes it.

- The Docker volume preserves the local database after restarts.

- create_tables.py creates the SQL tables. The vector table is configured when the vector store is first used.

- Some test_*.py files are manual smoke scripts that need a running database, sample IDs, or network access. Automated tests use mocks for selected tool and workflow behavior.

Current boundaries

Paper discovery uses arXiv. PDF ingestion runs in the request and can be slow for larger papers. There is no systematic retrieval or routing benchmark, authentication, or production deployment setup yet. These are fair future-work points; do not claim measured results until evaluations are added and run.